

2D Compositional Networks: Approximate Analysis by Synthesis

Lecture 12, part 1

Notes for Probabilistic Models of the Visual Cortex

Alan Yuille

Johns Hopkins University

8 October, 2026

What this lecture covers

Generalization under occlusion

Generative model of features

Robustness to occlusion

Detection under occlusion

Three additional projects

Summary

Colours used in the figures

In the network diagrams drawn for these notes, colour has one meaning throughout. In the other drawings and in the charts, colours only tell elements apart, and the captions say what they mark. The photographs and result pictures come from the papers and keep the papers' own colours, which the captions explain. A caption that says “CGN p. N ” took its picture from page N of the earlier course slides on compositional generative networks (Yuille, CGN); the text calls them the CGN slides.

- ▶ **Purple** marks the feature maps computed by the deep network.
- ▶ **Yellow** marks the von Mises-Fisher kernels and the likelihoods they produce.
- ▶ **Blue** marks the model of the object: the class mixtures.
- ▶ **Orange** marks the model of the occluder, and also attacks and out-of-distribution test data.
- ▶ **Green** marks the output of the network: the class score. Gold marks the scores of the single components, and pink the occluder likelihoods.
- ▶ **Red** frames the part of the network that a frame is about.

Explicit parts can be switched off when they are occluded

Lecture 11 ended with visual concepts. A visual concept is a cluster of the feature vectors that a deep network computes at one layer; the feature vectors of a cluster come from image patches that look alike, and most clusters correspond to subparts of objects: a corner of a car window, the rim of a wheel. A semantic part, such as a wheel or a headlight, is then detected by compositional voting. Each visual concept that is found in the image casts a vote for the position of the part, at the offset where it usually lies relative to the part.

The voting behaves well under occlusion. A visual concept that is hidden by an occluder is simply absent, so it casts no vote, and the visible concepts can still outvote the background. The representation is explicit: each subpart has its own detector, and a missing subpart switches its contribution off.

Explicit parts can be switched off when they are occluded

The question that Lecture 11 left open is: can we extend this to classify objects? This lecture does so with 2D compositional networks (CompNets). A CompNet is a generative model of the features of a deep network: it says how likely each feature vector is at each position if the image shows a given object. Because it generates every position, it can also say which positions it fails to explain, and those are the positions where the object is occluded.

The figure is Lecture 11's picture of compositional voting under occlusion (Wang et al., 2017). The occluders are objects pasted onto the vehicle: a goat, a cat and a person on the truck; a bottle and a dog on the bicycle. The first column marks semantic parts with boxes; the second column shows the voting for one part (the wheel of the truck, the headset of the bicycle). Each numbered circle is a visual concept: green for a cue present in the image, red for a cue hidden by an occluder (the ground under the wheel, the brake of the bicycle), which gives no vote.

Explicit parts can be switched off when they are occluded

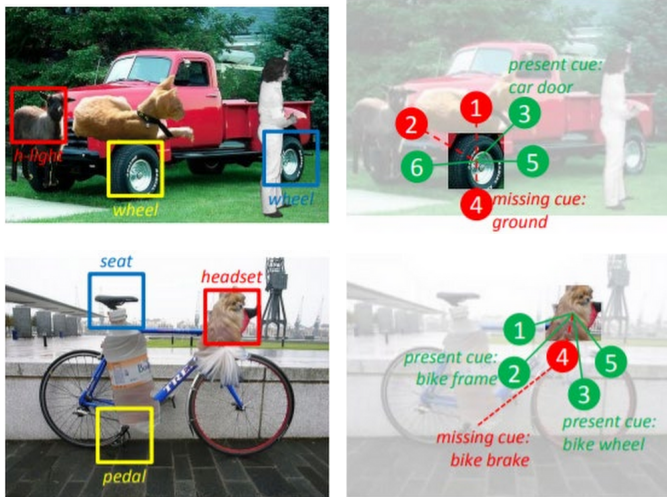


Figure 1: Compositional voting under occlusion (Wang et al., 2017; Lecture 11). First column: semantic parts. Second column: the visual concepts that vote for one part; green: present cues, red: missing cues.

Analysis by synthesis inverts a generative model of images

Analysis by synthesis formulates vision in a Bayesian perspective (Yuille and Kersten, 2006). The state of the world W collects everything that produced the image: which objects are present, their positions and viewpoints, the lighting, and anything that occludes them. Three distributions describe it.

- ▶ The generative model $P(I | W)$: a model for generating an image I from a state of the world W .
- ▶ The prior $P(W)$: how often each state of the world occurs.
- ▶ The posterior $P(W | I)$: how probable each state is once the image has been seen,

$$P(W | I) = \frac{P(I | W) P(W)}{P(I)}.$$

Synthesis means that we have models $P(I | W)P(W)$ for generating images. Analysis means that we can determine the world state W from the posterior $P(W | I)$, for example by finding the state of largest posterior probability.

Analysis by synthesis inverts a generative model of images

The figure gives an example. The world state is a car seen obliquely from the front, with two people and a horse in front of it. Synthesis renders the image from this description; analysis must recover the description from the image, including the fact that the horse and the people are in front of the car and hide most of it.

A full generative model of images has to produce every pixel: the texture of the paint, the reflections, the exact shape of the horse. Such models are very hard to learn, and most of these details are irrelevant to the question of which object is present. The CompNets of this lecture are generative on the features computed by the upper layers of a deep network, which are invariant to many low-level details of the image. This is why the title calls them approximate analysis by synthesis.

Analysis by synthesis inverts a generative model of images

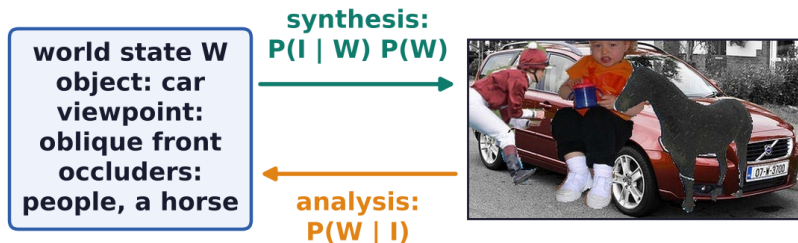


Figure 2: Analysis by synthesis. The box holds a state of the world W ; the image (a car with pasted occluders, from the Occluded-Vehicles dataset of Wang et al., 2017) is generated from it. The teal arrow is synthesis, with the generative model $P(I | W)$ and the prior $P(W)$; the orange arrow is analysis, with the posterior $P(W | I)$.

Out-of-distribution tests are tougher than i.i.d. splits

Usually vision models are tested by assuming that the data are independent, identically distributed (i.i.d.) samples from some unknown distribution. We collect a dataset, train on half of the data and test on the other half, and this yields standard performance measures such as the fraction of test images classified correctly.

The claim of this lecture is that these measures are problematic for many reasons. Datasets are finite, and they are usually balanced: each class has a similar number of images, and the images are typical ones. The average performance over such a test set says little about how a model behaves on rare cases, such as an object that is mostly hidden. A model can score well on average while failing badly on exactly the cases where vision is hard.

Tougher tests change the distribution between training and testing: out-of-distribution tests, where the test images come from a different distribution, and domain transfer, where a model trained on one domain is tested on another. In this lecture the test images contain occluders that never appear in training.

Out-of-distribution tests are tougher than i.i.d. splits



Figure 3: An i.i.d. split versus an out-of-distribution test. In the i.i.d. test (teal) the test images are new images of the training classes, taken in the same way: aeroplanes against the sky. In the out-of-distribution test (orange) the classes are the same, but the test images show the objects partly hidden by occluders, which the training images never contain.

Generalization under occlusion is important

In natural images objects are surrounded and partially occluded by other objects: a person in front of a car, a branch in front of a building, a sign on a pole in front of a taxi. Occlusion is the normal case.

Occluders are highly variable in terms of shape and texture, and they can cover any part of the object. This gives exponential complexity, and a vision system must generalize in exponentially complex domains: it must recognise objects under occlusion patterns that it has never seen.

A deep network learns what its training data show. If the training data contain few occluded objects, the network has little reason to recognise an object from a fraction of its parts, and its performance drops when the object is hidden.

Occlusion makes the number of possible images exponential

A simple count shows where the exponential complexity comes from. Divide the object into N regions, and suppose that an occluder can cover any subset of them. There are 2^N subsets, so 2^N patterns of visible and hidden regions: with $N = 20$ regions, more than a million. Each hidden region can, in addition, show any texture of any occluder: a person, a tree, a poster, a fence.

A training set can contain only a tiny fraction of these combinations, so a model that has to see each pattern during training cannot generalize to occlusion. A model that represents the object by its parts, and can decide part by part whether a part is visible, needs only to learn the parts: the combinations then come for free. This is the idea behind compositional models.

Generalization under occlusion is important

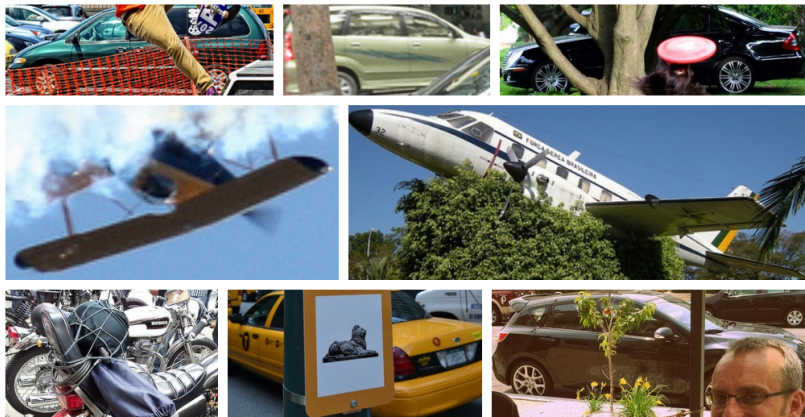


Figure 4: Objects in natural images from the MS-COCO dataset (Microsoft Common Objects in Context), each partly hidden. First row: cars behind an orange fence and a jumping person, behind a tree trunk, behind a tree, a frisbee and a head. Second row: an aeroplane in smoke, an aeroplane behind trees. Third row: a motorbike under a bag and a helmet, a taxi behind a poster, a car behind plants and a man's head.

Occlusion degrades DCNNs trained on unoccluded data

A deep convolutional neural network (DCNN) for classification computes a feature map with its convolutional layers and maps it to class scores with fully connected layers. VGG-16 (Visual Geometry Group, 16 weight layers; Simonyan and Zisserman, 2015) is one such network.

The experiment of Kortylewski et al. (2020) uses vehicles of the MS-COCO dataset (Microsoft Common Objects in Context). The training images show unoccluded vehicles; in the second row they are augmented by CutPaste, which pastes artificial occluders onto them. The test images show vehicles with real occlusion, grouped into levels by the fraction of the object that is occluded: L0, no occlusion; L1, 20 to 40 %; L2, 40 to 60 %; L3, 60 to 80 %. Performance is the classification accuracy in percent.

occluded area (%)	L0: 0	L1: 20–40	L2: 40–60	L3: 60–80	mean
VGG-16, no occlusion in training	99.1	88.7	78.8	63.0	82.4
VGG-16, trained with CutPaste	99.3	92.3	89.9	80.8	90.6

Occlusion degrades DCNNs trained on unoccluded data

The first row shows the problem. On unoccluded vehicles VGG-16 classifies 99.1 % of the vehicles correctly; when 60 to 80 % of the vehicle is hidden it classifies only 63.0 %.

What if we train with lots of augmented data? CutPaste is a form of data augmentation: it artificially occludes the training images with the four kinds of artificial occluders of the Occluded-Vehicles dataset, a set of vehicle images with pasted occluders (Wang et al., 2017): white boxes, noise, textures and objects cut out of other images. The second row shows that this helps, from 63.0 to 80.8 % at the highest level, but a large drop remains. The pasted occluders are only a sample of the exponentially many occlusion patterns, and the real occluders of the test images differ from them.

CompNets take a different route: they put the knowledge that an object can be occluded into the architecture of the network, so that the training data need not contain examples of every occlusion pattern.

Occlusion degrades DCNNs trained on unoccluded data

test images: real occlusion



training images augmented by CutPaste



Figure 5: Images of the experiment (Kortylewski et al., 2020). First row: test images with real occlusion (a police car whose bonnet is covered by a flame pattern, a car behind two pedestrians, a car behind a person in a dark coat). Second row: training images augmented by CutPaste, with animals and plants pasted in front of the cars.

A generative model of neural feature activations

A CompNet starts from the convolutional layers of a deep network (the backbone), such as VGG-16. For an input image they compute a feature map F : an array of $H \times W$ positions p , with a feature vector f_p of D numbers at each position ($D = 512$ for the upper convolutional layers of VGG-16). Here H and W are the height and the width of the feature map; this W has no relation to the world state W of analysis by synthesis. Each feature vector describes a patch of the image, the receptive field of position p .

A standard DCNN classifies F with its fully connected classification head. A CompNet replaces this head with a generative model of the feature map: for each object class y it says how probable the whole feature map F is, $p(F | y)$. The model generates the feature map of an upper layer of the backbone and leaves the pixels of the image aside.

A generative model of neural feature activations

This is approximate analysis by synthesis. A model of the image would have to generate every pixel. The features at the upper levels of the deep network are invariant to low-level details of the image, such as small shifts, changes of lighting and fine texture; modelling these details is hard, and they are often irrelevant to the task. A model of the features therefore needs to describe only the structure that matters for recognising the object: which parts are where.

The price is that the model can only say what the features can express. It works at the resolution of the feature map, one position for every 16×16 pixels of the image at the fourth pooling layer of VGG-16 (pool4) and every 32×32 at the fifth (pool5); the papers use one or the other, depending on the experiment. And it inherits the invariances of the backbone. Both points matter for fine-grained classes, such as speed-limit signs that differ only in their digits.

A generative model of neural feature activations

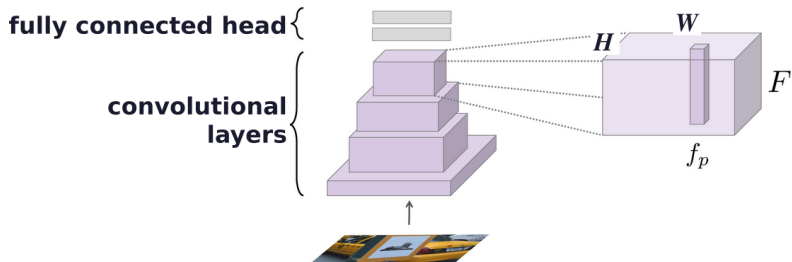


Figure 6: The feature map of a DCNN (CGN p. 8). The convolutional layers (the stacked blocks) turn the image at the bottom into the feature map F . The top face of the block is the $H \times W$ grid of positions; the vertical direction holds the D numbers of a feature vector, so the vertical bar is f_p at one position. The fully connected head (the two bars at the top) is what a CompNet replaces.

vMF kernels cluster the normalised feature vectors

Every feature vector is normalised to unit length, $\|f_p\| = 1$, so it is a point on the unit sphere in D dimensions. The model of a feature vector is built from von Mises-Fisher (vMF) distributions, the distributions on the sphere that Lecture 8 used for soft k-means:

$$p(f_p | \lambda_k) = \frac{e^{\sigma_k \mu_k^T f_p}}{Z(\sigma_k)}, \quad \|f_p\| = 1, \quad \|\mu_k\| = 1, \quad \lambda_k = \{\mu_k, \sigma_k\}.$$

The mean direction μ_k is a unit vector: the centre of the cluster. The concentration $\sigma_k > 0$ sets how tightly the feature vectors cluster around μ_k . The normaliser $Z(\sigma_k)$ makes the density integrate to one over the sphere; it depends on σ_k and D only,

$$Z(\sigma) = \frac{(2\pi)^{D/2} I_{D/2-1}(\sigma)}{\sigma^{D/2-1}},$$

where $I_{D/2-1}$ is the modified Bessel function of order $D/2 - 1$ (Banerjee et al., 2005). A CompNet uses K such distributions, the vMF kernels $\lambda_1, \dots, \lambda_K$, shared by all object classes.

The vMF density depends only on the angle to the mean

For unit vectors, $\mu_k^T f_p = \cos \phi$, where ϕ is the angle between f_p and μ_k . So the density is $e^{\sigma_k \cos \phi} / Z(\sigma_k)$: largest at $\phi = 0$, and the same for all feature vectors at the same angle from the mean. It is a Gaussian on the sphere: for small angles $\cos \phi \approx 1 - \phi^2/2$, so

$$e^{\sigma \cos \phi} \approx e^{\sigma} e^{-\sigma \phi^2/2},$$

a Gaussian in the angle with variance $1/\sigma$.

The density falls to half its peak where $\sigma(1 - \cos \phi) = \log 2$. For $\sigma = 2$ this is at $\phi = 49^\circ$, for $\sigma = 12$ at 20° , and for $\sigma = 30$, the value used in the CompNet papers, at 12° . These angles describe the density, not where samples fall. In $D = 512$ dimensions almost all of the sphere lies near 90° from any direction, and samples from a kernel with $\sigma = 30$ have a mean cosine of only about 0.06 with μ_k . The kernel still acts as a detector: its likelihood rises steeply with $\mu_k^T f_p$, and the kernels are compared with each other.

The figure shows the case $D = 2$, where the sphere is a circle and the vMF distribution is the von Mises distribution of an angle.

vMF kernels cluster the normalised feature vectors

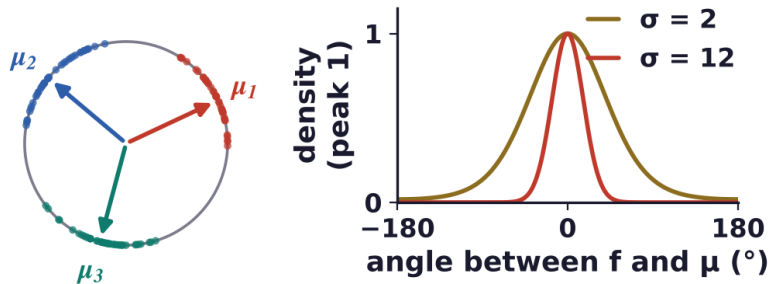


Figure 7: vMF kernels in two dimensions (computed). The circle holds unit vectors; the three arrows are mean directions μ_1 , μ_2 , μ_3 (at 25° , 140° and 255°), and the dots are 40 samples from each kernel with $\sigma = 12$. The plot gives the density against the angle between f and μ , for $\sigma = 2$ (olive) and $\sigma = 12$ (red), each scaled to a peak of 1.

The object model mixes viewpoints and, per position, parts

The model of class y is built in three levels:

$$p(F | \Theta_y) = \sum_m \nu^m p(F | \theta_y^m), \quad \nu^m \in \{0, 1\}, \quad \sum_m \nu^m = 1,$$

$$p(F | \theta_y^m) = \prod_p p(f_p | A_{p,y}^m, \Lambda), \quad p(f_p | A_{p,y}^m, \Lambda) = \sum_k \alpha_{p,k,y}^m p(f_p | \lambda_k).$$

Here y labels the object class, m labels the mixture component (viewpoint), p labels the position in F and k the vMF kernel.

- ▶ Last formula: at position p the feature vector comes from a mixture of the K kernels. The weights $\alpha_{p,k,y}^m \geq 0$, which sum to one over k , say which kernels (parts) are expected at p ; $A_{p,y}^m$ collects them, and Λ collects all the kernels.
- ▶ Middle formula: given the component m , the positions are independent, and the feature map is the product over positions. $\theta_y^m = \{A_{p,y}^m\}_p$ is the component.
- ▶ First formula: the class is a mixture of M components, and the binary ν^m selects exactly one of them. $\Theta_y = \{\theta_y^m\}_m$.

The object model mixes viewpoints and, per position, parts

The components are learnt without viewpoint labels. A single spatial arrangement of parts cannot describe a car seen from the side and from the front, so the training images of a class are split among M components by clustering their patterns of vMF likelihoods, and each component gets its own map of mixture weights; Kortylewski et al. (2020) use $M = 4$. For bicycles the images of one component end up sharing a viewpoint; for aeroplanes they share a background.

The α 's and the μ 's are parameters which are learnt; the concentrations σ_k are fixed (to 30). The kernels are shared by all classes and components, so a new class adds only mixture weights: $M \times H \times W \times K$ numbers.

Two simplifications keep the model cheap. Positions are independent given the component, so the model has no explicit springs between parts (terms that hold two parts at a preferred distance from each other); the spatial structure is in the position-specific weights $A_{p,y}^m$. And each position has one mixture, so the model is a spatial template of part probabilities, one template per viewpoint.

The log-likelihood is a sum over positions

Taking the logarithm turns the product over positions into a sum:

$$\log p(F | \theta_y^m) = \sum_p \log \left[\sum_k \alpha_{p,k,y}^m p(f_p | \lambda_k) \right].$$

Each position contributes one term: the log-probability of its feature vector under the parts expected there. A position whose feature vector matches none of the expected kernels contributes a large negative number, and this is what an occluder does to the score of the object model. The outlier model is designed to absorb such terms.

The binary selector ν^m turns the mixture over components into a choice. With the hard assignment of Lecture 8, the sum over components is replaced by its largest term, and the class is scored by the component that explains the feature map best:

$$s_y = \max_m \log p(F | \theta_y^m).$$

Each image is explained by one viewpoint.

Inference as a feed-forward neural network

Classification computes the class score of every class and picks the largest. All the steps are layers of a neural network.

1. vMF likelihood: $p(f_p | \lambda_k) = e^{\sigma_k \mu_k^T f_p} / Z(\sigma_k)$

2. mixture likelihoods: $p(F | \theta_y^m) = \prod_p \sum_k \alpha_{p,k,y}^m p(f_p | \lambda_k)$

3. class score: $s_y = \max_m s_y^m, \quad s_y^m = \log p(F | \theta_y^m)$

Step 1 is a 1×1 convolution of F with the kernels μ_k : at every position it computes the K dot products $\mu_k^T f_p$. The non-linearity $N(\cdot)$ then applies the exponential and the factor $1/Z(\sigma)$. The result L holds the likelihood of every kernel at every position, a block of size $H \times W \times K$.

Inference as a feed-forward neural network

The figure draws the network in two rows. Row one: the image passes through the DCNN backbone (purple) to the feature map F ; the vMF kernels $\mu_1, \dots, \mu_K$ (yellow) and $N(\cdot)$ give the likelihoods L . Row two: L goes to the class mixtures $A_y^1, \dots, A_y^M$ (blue), which give the scores $s_y^1, \dots, s_y^M$ (gold); their maximum is the class score s_y (green).

Step 2 combines L with the mixture weights of each class and component: a weighted sum over the kernels at each position, then a logarithm and a sum over positions. Step 3 takes the maximum over the components. The predicted class is the one with the largest score, and for training the scores are passed through a softmax.

The paper defines the scores as likelihoods; these notes write $s_y^m = \log p(F | \theta_y^m)$ because the logarithm turns the product into a sum and does not change which component or class is largest.

Because every step is differentiable (the maximum almost everywhere), the gradient of a loss can be propagated back through the generative model into the backbone, and the whole network can be trained end to end by backpropagation (Lecture 10).

Inference as a feed-forward neural network

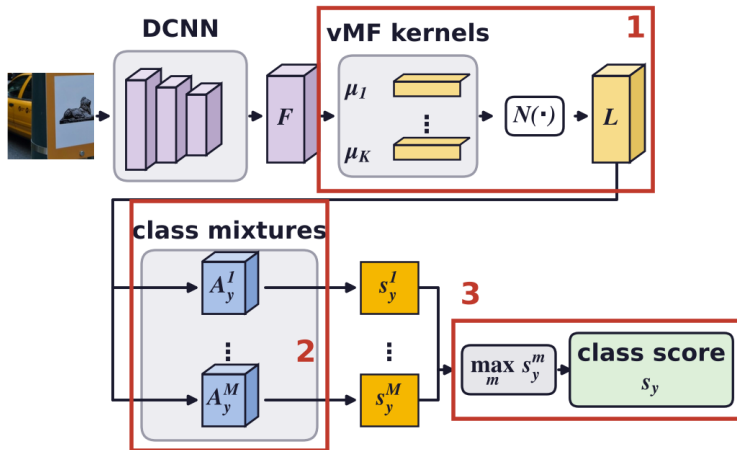


Figure 8: The CompNet as a feed-forward network (Kortylewski et al., 2020, Fig. 2, redrawn). Red boxes: the three steps.

Learning the model parameters with backpropagation

The network is trained on images with class labels by minimising

$$L = L_{\text{class}}(y, y') + \gamma_1 L_{\text{weight}}(W) + \gamma_2 L_{\text{vmf}}(F, \Lambda) + \gamma_3 L_{\text{mix}}(F, A_y).$$

The letter L is the loss here; it has no relation to the likelihood block L of the network. The four terms:

- ▶ L_{class} : the cross-entropy of the network output y' against the true class label y , as for any classifier.
- ▶ L_{weight} : weight decay, a penalty on the squared size of the backbone's weights, written W here (the paper writes ω); this W is the third use of the letter.
- ▶ L_{vmf} : makes the feature vectors cluster around the kernels.
- ▶ L_{mix} : makes the mixture model of the true class explain the feature map; A_y collects the mixture weights of class y .

The constants $\gamma_1, \gamma_2, \gamma_3$ weigh the terms. The last two are generative: they keep the CompNet a model of the features while the first term makes it discriminative.

With fixed concentration the vMF loss is a clustering loss

The vMF loss rewards each feature vector for being close to its nearest kernel:

$$L_{\text{vmf}}(F, \Lambda) = - \sum_p \max_k \log p(f_p | \mu_k).$$

With all concentrations fixed to the same σ , the normaliser $Z(\sigma)$ is a constant, and $\log p(f_p | \mu_k) = \sigma \mu_k^T f_p - \log Z(\sigma)$. The maximum over k picks the kernel with the largest dot product, so

$$L_{\text{vmf}}(F, \Lambda) = -\sigma \sum_p \max_k \mu_k^T f_p + \text{const.}$$

Minimising it maximises the cosine similarity between each feature vector and its nearest kernel: the objective of k-means on the sphere. The gradient moves each kernel towards the feature vectors assigned to it, and, through the backbone, each feature vector towards its kernel; so features and kernels move together.

Learning the model parameters with backpropagation

The mixture loss is the negative log-likelihood of the feature map under the best component $\hat{m}$ of the true class y , the component chosen in the forward pass:

$$L_{\text{mix}}(F, A_y) = - \sum_p \log \left[\sum_k \alpha_{p,k,y}^{\hat{m}} p(f_p \mid \lambda_k) \right].$$

It trains the mixture weights of that component to explain the feature maps of its images, as maximum likelihood would. In the paper this loss also skips the positions that the outlier model, introduced with occlusion, assigns to the occluder.

Training proceeds in two stages. The kernels are first initialised by clustering the feature vectors of the training images, and the mixture weights by maximum likelihood; then all parameters, including the backbone, are fine-tuned end to end by backpropagation with the full loss, σ staying fixed.

vMF kernels resemble part detectors

What does a kernel respond to? For each kernel μ_k , search the training images for the positions whose feature vectors have the highest likelihood under it (the largest $\mu_k^T f_p$), and show the image patches in the receptive fields of those positions.

The figure does this for four kernels. The patches of each kernel show the same kind of part, seen in different images and from different angles: aeroplane engines, fuselages with lettering, motorbike seats, motorbike wheels. In this sense the kernels resemble part detectors, although nothing in the training asked for parts.

The kernels are shared by all classes and viewpoints; the α 's place them in each object. Explainability follows: a detection can be explained by the parts which are present, and by where they are.

vMF kernels resemble part detectors

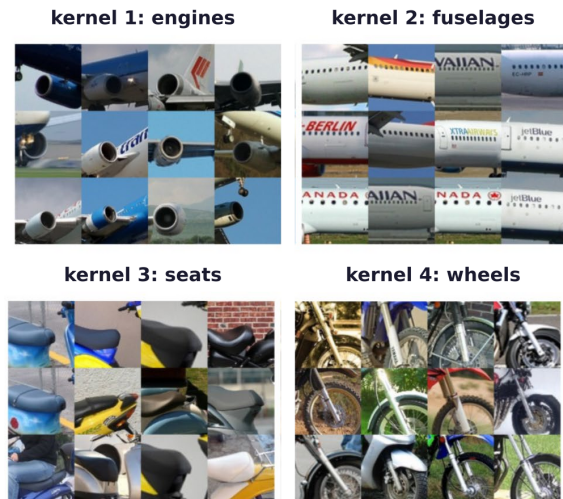


Figure 9: Image patterns with the highest likelihood for four vMF kernels (Kortylewski et al., 2021, from CGN p. 12). Each block of 4×3 patches belongs to one kernel; the names are read off the patches.

Mixture components model object pose

The mixture components can be inspected in the same way. Each training image of the bicycle class is assigned to the component with the highest likelihood for its feature map. The figure shows, for each of the four components, the training images with the highest likelihood, and below them a histogram of the 3D viewpoints of all images assigned to the component. The viewpoints are annotated in the dataset (PASCAL3D+, a dataset with 3D pose annotations; Xiang et al., 2014); they are used to draw the histograms and never used in training.

The histograms are polar plots of the azimuth, the direction from which the object is seen, in degrees around the circle. Each wedge is a range of azimuths; its length, and also its colour (from blue for few images to yellow and orange for many), gives the number of images.

Components 1, 2 and 3 each collect one dominant viewpoint, at about 270° , 90° and 0° ; component 4 collects the remaining views, spread over many directions. Images with different 3D viewpoint are approximately separated into different components.

Mixture components model object pose

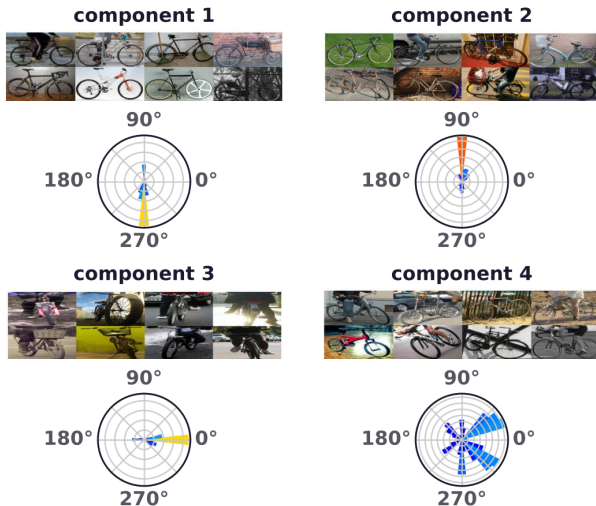


Figure 10: Bicycles: images with the highest likelihood and azimuth histogram of each component (Kortylewski et al., 2021, from CGN p. 15; bars traced and redrawn).

Aeroplane components also follow the background

The same inspection for aeroplanes gives a different picture. Components 1 and 2 hold aeroplanes on the ground, in front of airports and runways; components 3 and 4 hold aeroplanes in the sky. The azimuth histograms of components 1, 3 and 4 spread over many directions; that of component 2 is concentrated in one direction, at about 270° .

So the split follows the background more than the viewpoint: the components also capture the context. This is natural for a model of the whole feature map. The positions around the aeroplane hold features of sky or tarmac, and these differ more between the two groups of images than the views of the aeroplane do.

For classification this is harmless, perhaps even useful. For detecting an occluded aeroplane it is a problem, because the context can then outweigh the object.

Aeroplane components also follow the background

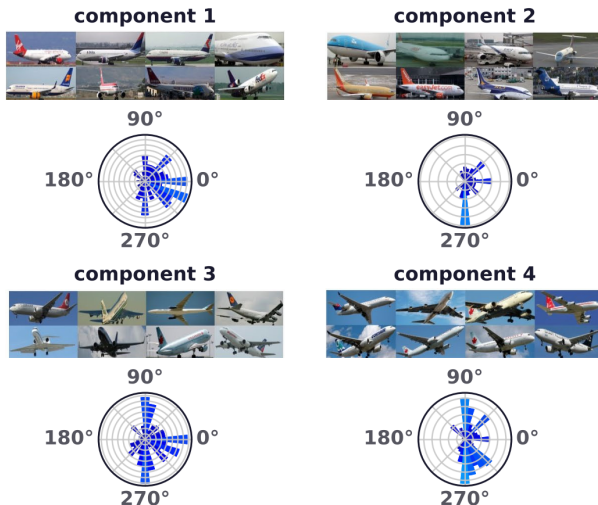


Figure 11: Aeroplanes: images with the highest likelihood and azimuth histogram of each component (Kortylewski et al., 2021, from CGN p. 16; bars traced and redrawn).

An outlier model explains the occluded positions

The object model of a class says which parts are expected at each position. At an occluded position the feature vector shows the occluder, so it fits none of the expected parts, and the product over positions is dragged down by every such position. The remedy is to let each position be explained either by the object or by an outlier model of “everything else”. A binary variable z_p^m records which: $z_p^m = 1$ if the feature vector at p is generated by the occluder, and $z_p^m = 0$ if it is generated by the object.

$$p(F | \theta_y^m, \beta) = \prod_p p(f_p, z_p^m=0)^{1-z_p^m} p(f_p, z_p^m=1)^{z_p^m}, \quad z_p^m \in \{0,1\},$$

$$p(f_p, z_p^m=1) = p(f_p | \beta, \Lambda) p(z_p^m=1),$$

$$p(f_p, z_p^m=0) = p(f_p | A_{p,y}^m, \Lambda) (1 - p(z_p^m=1)).$$

The exponents select one factor at each position: the occluder term where $z_p^m = 1$ and the object term where $z_p^m = 0$. For given z_p^m this is the joint probability of F and the z_p^m ; the paper writes it as $p(F | \theta_y^m, \beta)$.

An outlier model explains the occluded positions

The occluder model $p(f_p | \beta, \Lambda)$ uses the same vMF kernels as the object, with weights β that ignore the position:

$$p(f_p | \beta, \Lambda) = \sum_k \beta_k p(f_p | \lambda_k).$$

An occluder can appear anywhere, so the same mixture is used at every position. In practice there are several such models: Kortylewski et al. (2020) use five weight vectors $\beta_1, \dots, \beta_5$, and each position takes the one that explains it best.

The prior $p(z_p^m=1)$, the probability that a position is occluded, is fixed in advance, the same for all positions. It plays the role of the prior in Bayes decision theory: the larger it is, the more readily a position is declared occluded.

The CGN slides call the outlier model a simple model of what the object does not look like. It needs no knowledge of the particular occluder; it only has to explain feature vectors that the object model explains poorly.

With the outlier model the class score takes the better term

The variables z_p^m are not observed, so they are inferred: at each position, z_p^m picks the larger of the two terms. Because the positions are independent given the component, the maximum over all z_p^m splits into a maximum at each position:

$$\max_{\{z_p^m\}} \log p(F \mid \theta_y^m, \beta) = \sum_p \max \left\{ \log p(f_p, z_p^m=0), \log p(f_p, z_p^m=1) \right\}.$$

This is the class score of a component with the outlier model: each position contributes the log-likelihood of the object model, or that of the occluder model if it is larger. An occluded position now costs at most the occluder's log-likelihood, a moderate number, and the visible positions decide the score.

In the network this adds one branch: the occluder likelihoods O , computed from the same block L of kernel likelihoods, and a maximum with the object likelihoods at every position. The set of values of z_p^m over the positions is the occlusion map Z^m of component m (a map, unrelated to the normaliser $Z(\sigma)$); it is returned together with the class score.

The outlier model is learnt from images without the objects

The occluder weights β are learnt from clustered features of random natural images that do not contain any object of interest: beaches, meadows, roads, streets. The feature vectors of these images are computed by the backbone and described with the vMF kernels, and the mixtures of kernels that fit them form the occluder models.

The occluder models are learnt once and do not depend on the class, because background clutter looks the same whatever object it hides. And they are learnt without any image of an occluded object, so the network can be trained on unoccluded objects and still explain occluders at test time.

The figure shows where the occluder branch enters the network. The occluder models $\beta_1, \dots, \beta_5$ (orange) take the kernel likelihoods L and give the occlusion likelihoods O (pink) at every position. Every mixture component compares them with its own likelihoods and outputs two things: its class score s_y^m and its occlusion map Z^m . The class score of the best component is the class score s_y , and its occlusion map Z shows where the outlier model won.

The outlier model is learnt from images without the objects

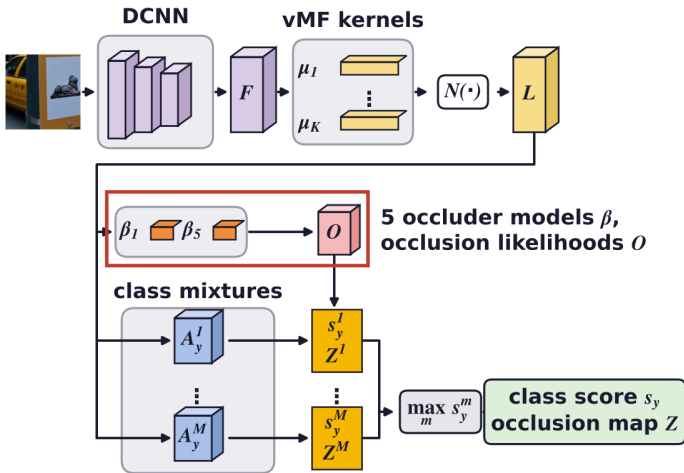


Figure 12: The CompNet with the outlier model (Kortylewski et al., 2020, Fig. 2, redrawn). Red box: the new branch.

Occluder models are clusters of background features

The figure shows examples of the data from which the occluder models are learnt (CGN p. 19). They contain none of the object classes, so every feature vector they produce is, from the point of view of the object models, an outlier. The occluder models therefore learn what background clutter looks like in feature space: grass, sky, water, foliage, walls.

Occluders in test images are often objects of other classes, such as people or animals, which are not in these background images. They are still explained reasonably well, because their feature vectors are far from the parts that the object model expects at those positions, and the occluder model needs only to beat the object model, not to describe the occluder accurately.

Occluder models are clusters of background features



Figure 13: Natural images that contain none of the objects of interest, of the kind used to learn the occluder models (from CGN p. 19): a beach, a meadow with a bench, a road through autumn trees, a town street at dusk.

The object and outlier models compete at every position

The choice of z_p^m is a decision between two hypotheses for one feature vector:

$$z_p^m = 1 \quad \text{if} \quad p(f_p, z_p^m=1) > p(f_p, z_p^m=0).$$

Writing $\pi = p(z_p^m=1)$ and taking logarithms gives the form of the Bayesian test of Lecture 4:

$$z_p^m = 1 \quad \text{if} \quad \log \frac{p(f_p | \beta, \Lambda)}{p(f_p | A_{p,y}^m, \Lambda)} > \log \frac{1 - \pi}{\pi}.$$

The left side is a log-likelihood ratio: occluder against object. The right side is a threshold set by the prior. Lecture 4 applied the same test to filter responses, edge against non-edge; here it is applied to the feature vectors of a deep network.

The maps in the figure plot an occlusion score at each position, a measure such as the log-likelihood ratio of the test: large (bright) where the outlier model is much better, small (dark) where the object model is better. Thresholding the map gives the binary z_p^m .

The object and outlier models compete at every position

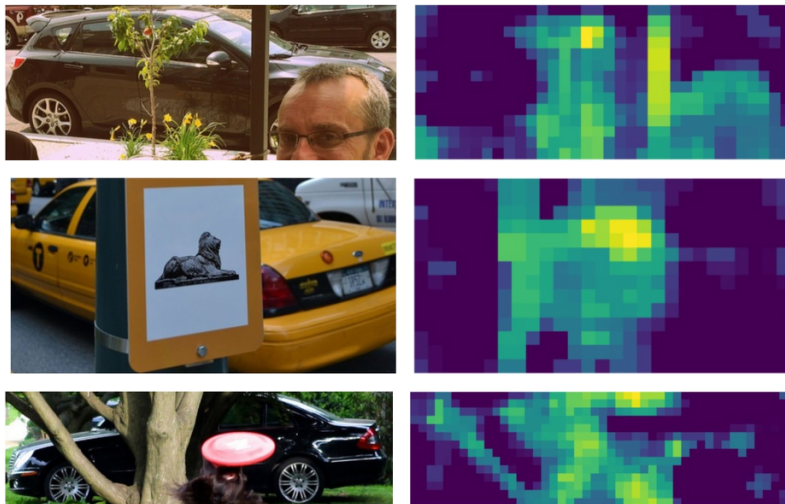


Figure 14: Images and their occlusion maps (Kortylewski et al., 2020, from CGN p. 20): a plant, a post and a man's head in front of a car; a sign in front of a taxi; a tree, a frisbee and a dog in front of a car. Bright: explained by the outlier model.

Occluded-Vehicles pastes four kinds of occluders onto vehicles

How well are occluders localized? The test needs images in which the occluded positions are known. The Occluded-Vehicles dataset provides them. Wang et al. (2017) built it from vehicles of PASCAL3D+, a dataset of images annotated with the 3D pose of the objects (Xiang et al., 2014), by pasting objects cut out of other images onto them; Kortylewski, Liu, Wang, Zhang and Yuille (WACV 2020) added white boxes, noise and textures. Because the occluders are pasted, their masks are known exactly.

There are four kinds of artificial occluders: white boxes, patches of random noise, patches of texture, and real objects cut out of other images. The occluders cover a controlled fraction of the object, at three levels: 20 to 40 %, 40 to 60 % and 60 to 80 %. White boxes and noise are easy to tell from an object; textures and natural objects are harder, because their features can resemble the features of object parts.

Occluded-Vehicles pastes four kinds of occluders onto vehicles

white boxes



noise



textures



objects



Figure 15: The four kinds of occluders of the Occluded-Vehicles dataset on the same car (from CGN p. 21): white boxes, noise, textures and natural objects (two people and a horse). The colour of each name is the colour of its curve in the localization results.

Occluders are localized well, natural objects least well

Occluder localization is a binary decision at each pixel: occluded or not. Each pixel is classified by comparing the occlusion score at its position with a threshold. A positive is an occluded pixel; a true positive is an occluded pixel that is flagged, and a false positive is an unoccluded pixel that is flagged.

The receiver operating characteristic (ROC) curve, from Lecture 3, plots the true positive rate (the fraction of occluded pixels that are flagged) against the false positive rate (the fraction of unoccluded pixels that are flagged) as the threshold moves. A low threshold flags everything (the point (1,1)), a high one flags nothing (the point (0,0)), and a good detector bends towards the top left.

The legend gives the true positive rate at a false positive rate of 0.2: 0.92 for white boxes, 0.88 for noise, 0.71 for textures and 0.64 for natural objects. At this operating point Kortylewski et al. (2020) report gains of 12, 19, 6 and 8 % in true positive rate over an earlier compositional model built on a dictionary of clustered features.

Occluders are localized well, natural objects least well

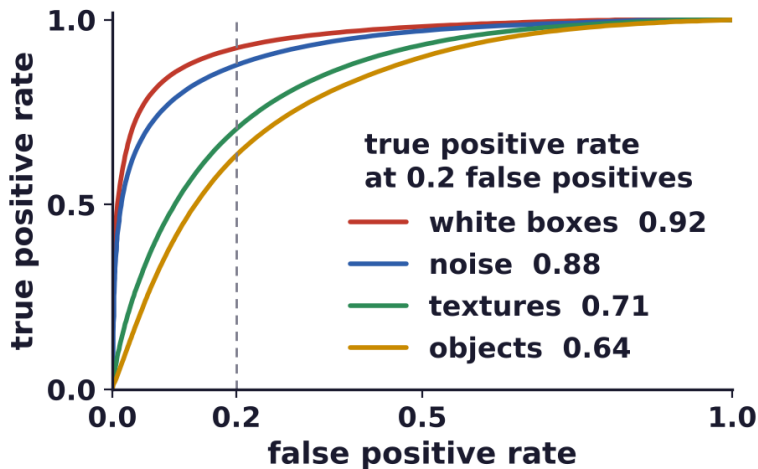


Figure 16: Occluder localization on Occluded-Vehicles, levels L1 to L3 (Kortylewski et al., 2020; curves traced from CGN p. 21 and redrawn). Dashed: false positive rate 0.2.

CompNets can classify partially occluded vehicles robustly

The main classification test (Kortylewski et al., 2021) is both out of distribution and a domain transfer. The networks are trained on vehicles of PASCAL3D+ without occlusion and tested on vehicles of MS-COCO with real occlusion, at the occlusion levels L0 to L3. The backbones are VGG-16, ResNet-50 (a residual network with 50 layers; Lecture 10) and ResNeXt (a residual network whose layers are split into parallel groups). Each CompNet replaces the classification head of the same backbone.

occluded area	L0	L1	L2	L3	mean
VGG-16	97.8	86.8	79.1	60.3	81.0
ResNet-50	98.5	89.6	84.9	71.2	86.1
ResNeXt	98.7	90.7	85.9	75.3	87.7
CompNet on VGG-16	98.2	89.1	84.3	78.1	87.5
CompNet on ResNet-50	98.5	92.6	88.9	83.6	90.9
CompNet on ResNeXt	98.8	94.0	93.5	87.7	93.5

Classification accuracy in percent; L0: 0 %, L1: 20–40 %, L2: 40–60 %, L3: 60–80 % of the object occluded.

CompNets can classify partially occluded vehicles robustly

On unoccluded vehicles (L0) the CompNets and the plain networks are equally accurate: replacing the classification head costs nothing. The gains grow with the occluded area. At L3, with 60 to 80 % of the vehicle hidden, the CompNet on VGG-16 reaches 78.1 % against 60.3 % for VGG-16, and the CompNets on ResNet-50 and ResNeXt gain about 12 points each. Built on the same backbones, CompNets gain most where most of the object is occluded.

The CompNets were trained without occluded images and without CutPaste augmentation. The outlier model is the part designed for robustness: it lets the visible parts decide the class. The table does not separate its contribution from that of the part-based model. The plain networks here are trained on PASCAL3D+, so their numbers differ from those of the MS-COCO-trained VGG-16 of the CutPaste experiment; the CompNet on ResNeXt uses 1024 kernels.

CompNets can classify partially occluded vehicles robustly



Figure 17: Test images with real occlusion from MS-COCO (from CGN p. 22): a police car whose bonnet is covered by a flame pattern, a car behind two pedestrians, a car behind a person in a dark coat, a red car behind a person in white.

CompNets also gain on 50 occluded ImageNet classes

ImageNet is a harder test: its classes are animals, plants and man-made objects of every kind, photographed in many poses and settings. The experiment uses 50 classes, tested with artificial occluders at the levels L1 to L3 as for the vehicles (Kortylewski et al., 2021). The baseline ResNeXt is trained with CutOut, which erases random squares of the training images.

occluded area	L0	L1	L2	L3	mean
ResNeXt, trained with CutOut	97.7	58.6	28.4	13.5	49.6
CompNet on ResNeXt	95.4	61.3	37.0	21.2	53.7

The CompNet gains at every level of occlusion, by 2.7, 8.6 and 7.7 points, and loses 2.3 points without occlusion. All accuracies under occlusion are much lower than for vehicles.

CompNets also gain on 50 occluded ImageNet classes

The CompNet is best suited to object classes whose appearance a few mixture components can represent. A vehicle seen from four viewpoints is such a class. A class such as “dog” or “chainsaw” is probably harder: its images vary in pose, scale, articulation and setting far more than four spatial templates of parts can describe, and the components then average over very different images.

This points to the limits of the model: a rigid spatial template per component, and no articulation. The CompNet authors name articulated objects as future work.

CompNets also gain on 50 occluded ImageNet classes

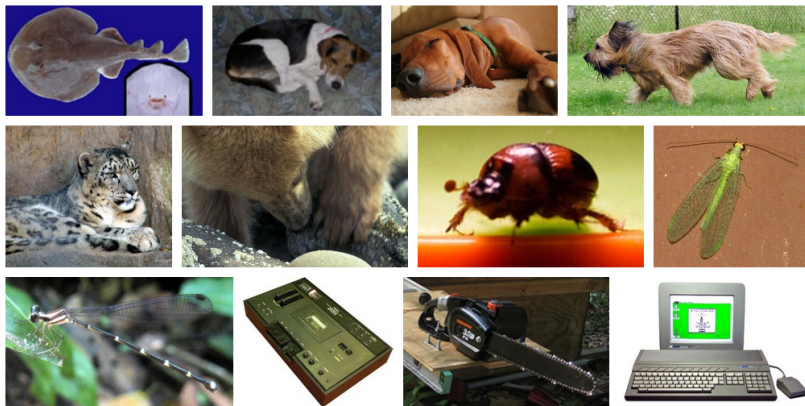


Figure 18: Examples of the 50 ImageNet classes, shown without occlusion (from CGN p. 23): animals (a ray, dogs, a snow leopard, a bear), insects (a beetle, a lacewing, a damselfly) and objects (a cassette player, a chainsaw, a computer).

DCNNs for object detection also generalize poorly

Classification says which object an image shows. Detection must also say where: it outputs a bounding box around each object, with a class label. A box is counted as correct if it overlaps the ground-truth box enough; the overlap is measured by the intersection over union (IoU), the area of the intersection of the two boxes divided by the area of their union, and the usual criterion is $\text{IoU} \geq 0.5$.

Faster R-CNN (Ren et al., 2015; R-CNN stands for regions with CNN features) is a standard DCNN detector. A region proposal network (RPN) suggests candidate boxes, the proposed regions, and a classification head scores the features inside each box and refines its coordinates.

Under occlusion such detectors fail in a characteristic way. In the figure a bicycle stands behind a fence. Faster R-CNN's box (red) is far too large: it includes the fence and much of the wall. The box of a context-aware CompNet (green), a CompNet that models the context separately from the object, is close to the ground truth (blue).

DCNNs for object detection also generalize poorly

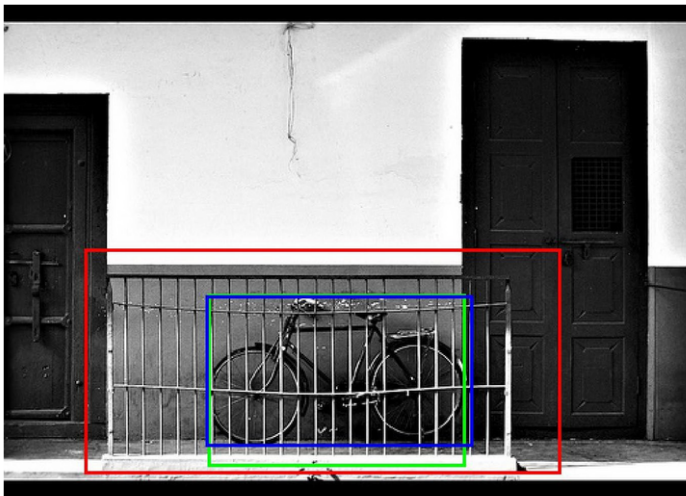


Figure 19: A bicycle behind a fence, from MS-COCO (Wang et al., 2020, Fig. 1, from CGN p. 26). Blue box: ground truth; red box: Faster R-CNN; green box: context-aware CompNet.

Context has too much influence when object is occluded

When the original CompNet is used to place boxes, its boxes go wrong in a different manner from Faster R-CNN's, and the reason is the context.

A CompNet is a model of the whole feature map inside a box around the object, and for aeroplanes most of that box is sky: the training aeroplanes are surrounded by sky. The mixture weights therefore expect sky features around the aeroplane, and the CompNet learns the sky as part of the aeroplane. This was already visible in the mixture components, which split aeroplanes on the ground from aeroplanes in the sky.

When the aeroplane is occluded, its own parts give weak evidence, and the context dominates. In the figure, the box of the original CompNet (orange) is placed by voting for the object centre and two corners from the positions of the feature map, with object and context weighted equally. The sky casts strong votes, the centre is pulled up, and the box covers the sky above the aeroplane.

Context has too much influence when object is occluded



Figure 20: An occluded aeroplane and three training images (Wang et al., 2020, Fig. 4, from CGN p. 27). In the large image people and a pickup truck stand in front of the aeroplane; blue box: ground truth; orange box: the original CompNet; green box: the context-aware CompNet. The three images on the side are training aeroplanes, surrounded by sky.

Context-aware CompNets separate context and object

The context-aware CompNet (Wang et al., 2020) gives each position two mixture models: $A_{p,y}^m$ for the object, as before, and $\chi_{p,y}^m$ for the context, with a fixed weight ω between them:

$$p(f_p | A_{p,y}^m, \chi_{p,y}^m, \Lambda) = \omega p(f_p | \chi_{p,y}^m, \Lambda) + (1 - \omega) p(f_p | A_{p,y}^m, \Lambda).$$

Both terms are mixtures of the same vMF kernels. The weight ω controls how much the context may count. With $\omega = 0.5$ the two terms have equal weight; Wang et al. (2020) state that this retains the original CompNet, which learnt object and context as one mixture. With $\omega = 0$ the context is ignored. This ω is unrelated to the network weights of other lectures.

Context-aware CompNets separate context and object

To learn two separate mixtures, the training images must be split into object and context. The feature vectors of the background regions of the training images (outside the boxes) are clustered with k-means++ (k-means with a careful random choice of the initial centres) into a dictionary of context features. Inside a training box, a position is labelled context if its feature vector is close to an entry of the dictionary (cosine similarity above a threshold), and object otherwise.

The context mixtures χ are then learnt from the context positions and the object mixtures A from the object positions, each by maximum likelihood as before. Only bounding boxes are needed: the segmentation into object and context comes from the data.

The figure shows such segmentations. In each mask, yellow positions are the object and purple positions the context. The masks are coarse, because they live on the grid of the feature map, but they follow the shape of the aeroplane, the car and the cyclist.

Context-aware CompNets separate context and object



Figure 21: Training images and their segmentations into object and context (Wang et al., 2020, from CGN p. 28): an aeroplane, a car and a bicycle with its rider. Yellow positions of the masks are object, purple positions context.

Bounding box voting finds the centre and two corners

The context-aware CompNet also changes how a box is found. Bounding box voting (BBV) learns additional mixture components that model the feature activations expected around three points of the box: the object centre and two opposite corners (top right and bottom left). Each model gives, at every position of the feature map, the likelihood that the point lies there.

The likelihood of each point is computed from the parts around it, so the visible parts vote for the centre and the corners even when other parts are hidden. Any two of the three points determine a box (the two corners, or the centre and one corner), and the box is generated from the two points that have maximal likelihood. A detector that has to see the whole object to place its box fails under occlusion; one that lets the visible parts vote does not need the hidden ones.

Context-awareness improves localization

The figure shows the probability map of the object centre for the occluded aeroplane, for two values of the context weight. With $\omega = 0.5$, the original CompNet, the high-probability region lies above the aeroplane: the sky pulls the centre up. With $\omega = 0.2$ the context counts less, and the centre lies on the aeroplane.

The value $\omega = 0.2$ keeps some context. Context is useful: an aeroplane is more likely in the sky or on a runway than in a living room. It is harmful only when it outweighs the object.

Context-awareness improves localization



Figure 22: The occluded aeroplane with its boxes and the probability maps of the object centre (Wang et al., 2020, Fig. 4, from CGN p. 29). Boxes: blue, ground truth; orange, $\omega = 0.5$ (the original CompNet); green, $\omega = 0.2$. The map labels have the colours of the boxes. Bright positions are likely centres.

The outlier model also localizes occluders in detection

The outlier model works inside a detector as it does in a classifier. Within each detected box, every position is explained by the object model, the context model or the outlier model, and the positions taken by the outlier model form an occlusion map.

The figure shows occlusion maps of the context-aware CompNet on MS-COCO images. Yellow positions are explained by the outlier model, dark ones by the object or its context; teal and green mark intermediate occlusion scores. People in front of the bus, a cyclist in front of the car, a pillar in front of the train, a pedestrian in front of a bicycle and a skateboarder in front of a car are found. The boxes on the images are blue for the ground truth and green for the CompNet, with the class label.

This is the explainability of the model: the detector says where the bus is, and also which parts of the bus it could not see.

The outlier model also localizes occluders in detection



Figure 23: Detections and occlusion maps on MS-COCO (Kortylewski et al., 2021, Fig. 12, from CGN p. 30); each image is followed by the occlusion map of its box. Yellow: outlier model.

Context-aware CompNets detect occluded vehicles best

The comparison (Wang et al., 2020) uses vehicles of MS-COCO, grouped by the amount of occlusion into five levels, L0 (lightest) to L4 (heaviest); the paper calls L0 and L1 light occlusion (2890 images) and L2 to L4 heavy occlusion (788 images). The measure is the average precision (AP) at IoU 0.5: the area under the precision-recall curve of the detector, in percent. Precision is the fraction of the detections that are correct, and recall the fraction of the objects that are found.

method	L0	L1	L2	L3	L4
Faster R-CNN	81.7	66.1	59.0	40.8	24.6
Faster R-CNN with reg. (CutOut)	84.3	71.8	63.3	45.0	33.3
Faster R-CNN with occ. (occluded training)	85.1	76.1	66.0	50.7	45.6
CompNet with BBV, $\omega = 0.5$	83.5	77.1	70.8	51.7	40.4
Context-aware CompNet, BBV, $\omega = 0.2$	88.7	82.2	77.8	65.4	59.6
Context-aware CompNet, BBV, $\omega = 0$	91.8	83.6	76.2	61.1	54.4

Context-aware CompNets detect occluded vehicles best

The three rows of Faster R-CNN differ in training: on unoccluded data; with CutOut regularisation (“reg.”); and with images of the same dataset occluded at all levels (“occ.”). Even the last, which has seen occluded vehicles, drops to 45.6 at L4.

The CompNets were trained without occlusion. With $\omega = 0.5$ (the original CompNet with bounding box voting) they are far better than plain Faster R-CNN at L1 to L4 (40.4 against 24.6 at L4) and comparable to Faster R-CNN trained on occluded images. Reducing the context weight helps most under heavy occlusion: with $\omega = 0.2$ the AP at L4 is 59.6. Ignoring the context entirely ($\omega = 0$) is best at L0 and L1, and 1.6 to 5.2 points below $\omega = 0.2$ at L2 to L4, where a little context helps to find a mostly hidden object.

The figure shows three detections. The yellow box is a CompNet that classifies the regions proposed by the RPN of Faster R-CNN; it inherits the proposals' errors, which is why bounding box voting is needed.

Context-aware CompNets detect occluded vehicles best

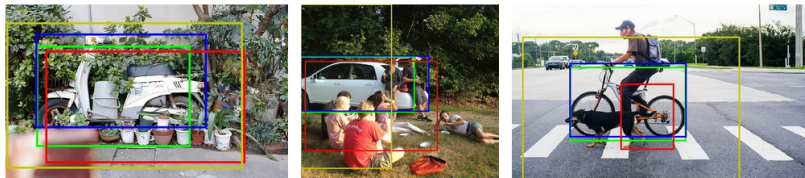


Figure 24: Detections on occluded vehicles (Wang et al., 2020, Fig. 2, from CGN p. 31): a motorbike behind potted plants, a car behind people sitting on the grass, a bicycle and its rider behind a dog. Blue box: ground truth; red: Faster R-CNN; yellow: CompNet on the proposed regions; green: context-aware CompNet with bounding box voting.

Prior knowledge in the network narrows the complexity gap

Partial occlusion introduces exponential complexity in the data: the number of occlusion patterns grows exponentially with the number of parts, so no dataset covers them. This is the complexity gap between what a network can see in training and what it meets in testing.

The gap can be overcome by introducing prior knowledge about compositionality, partial occlusion and context into the neural architecture, instead of learning it from data:

- ▶ compositionality: an object is a spatial arrangement of parts (the vMF kernels and the mixture weights);
- ▶ partial occlusion: each position can be explained by an outlier model (the variables z_p^m);
- ▶ context: the context is modelled separately and weighted (ω).

The result is generalization beyond the training data in terms of partial occlusion and context, while high discriminative performance is retained thanks to end-to-end training. The gap narrows; it does not vanish: accuracy still falls as more of the object is hidden.

Prior knowledge in the network narrows the complexity gap

The future work listed by the CompNet authors: articulated objects (whose parts move relative to each other, such as people and animals), 3D geometry (models of the object in 3D instead of one 2D template per viewpoint), top-down reasoning (using the interpretation of the whole image to revise local decisions) and scale.

Three further projects take up some of these points. They test CompNets against adversarial patches, extend them to the shape of the occluded object, and add top-down reasoning about which of several objects occludes which.

Prior knowledge in the network narrows the complexity gap



Figure 25: The taxi behind a poster and its occlusion map (from CGN p. 20): the outlier model explains the poster, and the visible parts of the taxi decide the class.

Patch attacks fool deep nets with a few textured patches

Standard practice in computer vision and machine learning evaluates an algorithm by its average performance on a finite, balanced, annotated dataset. A tougher evaluation looks for the weak points of the algorithm by dynamic testing: the input images are modified adaptively, in response to the algorithm's outputs, until the algorithm fails. The CGN slides call such a tester an adversarial examiner.

Lecture 11 introduced PatchAttack (Yang et al., 2020), an adversarial examiner for classifiers. It pastes a few small patches of texture onto the image. The attack is black-box: it sees only the outputs of the network, never its weights or gradients. And it is targeted: the attacker chooses a target class, and the attack succeeds if the network outputs that class.

These black-box targeted attacks had over 90 % success on advanced deep networks, while covering only a small part of the image. This suggests that deep nets lack knowledge of the spatial structure of objects: a few patches with the texture of another class, placed anywhere, are enough to change the decision.

Patch attacks fool deep nets with a few textured patches

PatchAttack learns an attack policy by reinforcement learning. The agent (RL agent) repeats a loop of six steps:

1. select the position of a patch in the input image;
2. select a texture image from a dictionary of textures for the target class (textures computed from images of that class);
3. select the position of the patch in the texture image;
4. cut the patch out of the texture and paste it at the chosen position;
5. feed the modified image to the network;
6. receive a reward from the network's output: large if the output moves towards the target class.

The policy is updated from the rewards, so later choices of position and texture become better. Several patches are placed in turn.

Patch attacks fool deep nets with a few textured patches

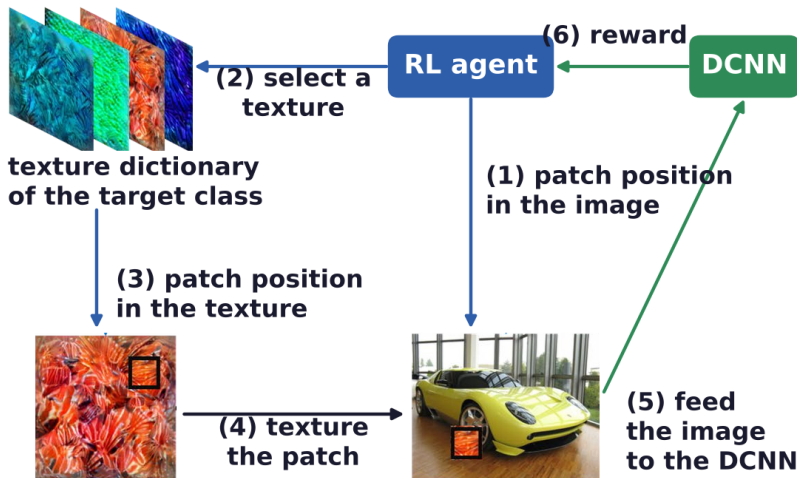


Figure 26: PatchAttack (Yang et al., 2020, Fig. 1, from CGN p. 35, redrawn); the numbers are the six steps of the loop.

CompNets are robust to patch attacks out of the box

Cosgrove et al. (2020) conjectured that CompNets would do better than deep nets, for two reasons: they have knowledge of the spatial structure of objects, so a patch of texture in the wrong place does not fit the object model; and their outlier model may enable them to reject the attacking patches, by explaining them as occluders. “Out of the box” means that the CompNets were not trained against attacks.

Two black-box attacks are used, each allowed to cover at most 10 % of the image (PASCAL3D+):

- ▶ the texture patch attack (TPA): PatchAttack, with 1 or 4 patches;
- ▶ Sparse-RS (Croce et al., 2022): a random search over the positions and the pixel values of a patch, with 1 patch, which needs only the network’s outputs.

The measure is the attack success rate: the percentage of images on which the attack succeeds. Lower is better for the network.

CompNets are robust to patch attacks out of the box

targeted attacks	accuracy	TPA, 1 patch	TPA, 4 patches	Sparse-RS
VGG-16	98.8	52.6	88.0	84.7
VGG-16 + adversarial training	96.0	8.6	53.3	33.5
CompNet on VGG-16	98.2	2.4	8.2	5.8

untargeted attacks	TPA, 1 patch	TPA, 4 patches	Sparse-RS
VGG-16	91.6	95.4	99.6
VGG-16 + adversarial training	34.2	79.5	75.4
CompNet on VGG-16	7.8	24.9	18.0

Accuracy without attack and attack success rates, in percent; Sparse-RS with one patch (Cosgrove et al., 2020, Table 1). An untargeted attack succeeds if the output is any wrong class. Adversarial training trains the network on attacked images.

CompNets are robust to patch attacks out of the box

Patch attacks are less successful on CompNets by an order of magnitude than on VGG-16: for targeted attacks, 8.2 against 88.0 % with four TPA patches, and 5.8 against 84.7 % for Sparse-RS. Against the adversarially trained VGG-16, which has seen attacks during training, the CompNet is still about six times harder to attack (53.3 against 8.2, 33.5 against 5.8). Untargeted attacks are easier, since any wrong class counts, and the ratios are smaller (95.4 against 24.9 for four TPA patches), but the ordering is the same.

CompNets can also localize the attacks: the patches appear in the occlusion map. In the figure the attacked image has several textured patches on the car, and the occlusion map lights up at their positions. The colours of the maps are those of Cosgrove et al.: dark positions are explained by the object model, and brighter colours mark higher occlusion scores, that is, positions that the CompNet takes for occluders or background. The few bright cells at the corners of the unattacked map are background.

CompNets are robust to patch attacks out of the box

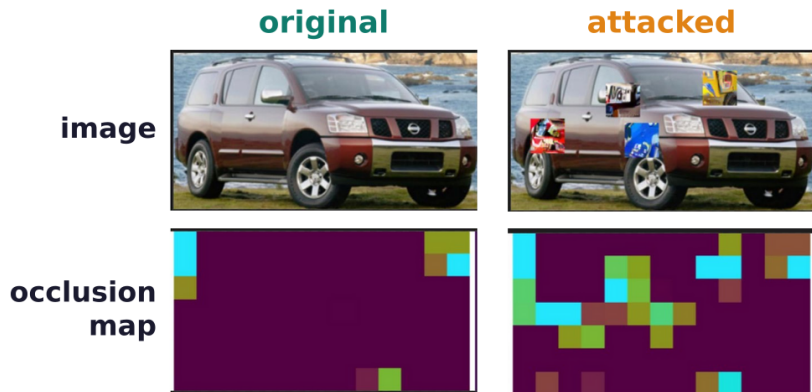


Figure 27: An SUV before and after a patch attack, with the occlusion maps of the CompNet (Cosgrove et al., 2020, from CGN p. 37). The patches of the attacked image appear as bright positions in its occlusion map.

CompNets need finer detail for similar traffic signs

The same tests on the German Traffic Sign Recognition Benchmark (GTSRB; Stallkamp et al., 2012), with 1000 test images, untargeted attacks with one patch of at most 1 % of the image (Cosgrove et al., 2020):

model	accuracy	TPA success	Sparse-RS success
VGG-16	95.2	90.1	92.6
CompNet on VGG-16	61.7	23.7	43.9
CompNet, fine-tuned per position	75.9	40.6	64.2
+ two stages with VGG-16	93.0	64.4	75.8

The CompNet is far harder to attack, but its accuracy without attack is only 61.7 %. Its robustness to the attacks was better than that of the alternatives, particularly for signs that differ at the coarse level.

CompNets were less effective for fine-scale discrimination: speed-limit signs share the overall shape and colour and differ only in their digits, for example 60 against 80.

CompNets need finer detail for similar traffic signs

Cosgrove et al. used an engineering trick to partially solve this problem, in two steps. Part-based fine-tuning trains the CompNet's features to be predictive of the class at each position: a simple linear classifier predicts the class from each feature vector, and the predictions are max-pooled over the positions. The two-stage model first classifies the image with VGG-16, and only predicts with the CompNet if VGG-16's confidence (its largest softmax probability) drops below a threshold (0.95). Accuracy returns to 93.0 %, at the price of part of the robustness.

The deeper reason lies in what CompNets model: they are generative on features at the upper levels of the deep net, which are invariant to low-level details of the image. This is a big advantage when the details are irrelevant, and a handicap when the details are the task. It suggests that the coarse CompNets need to be supplemented by fine-scale CompNets defined at lower levels of the network, with more detailed geometry.

Bottom-up processing seeks invariance; top-down adds detail

The same division of labour appears in a theory of the visual cortex (Lee and Mumford, 2003). Bottom-up processing, from the image towards higher areas, seeks invariant features: features that stay the same when the object moves, turns or is lit differently, and that are therefore good for recognising it. Invariance throws away detail, such as the exact position of an edge or the shape of a digit.

Top-down processing, from higher areas back towards the lower ones, is needed for higher resolution: once a hypothesis has been formed (a speed-limit sign), the lower areas, which still hold the detail, can be consulted to decide between the alternatives (60 or 80). A coarse CompNet on high-level features followed by a fine CompNet on low-level features would be a computational version of this loop.

CompNets find modal and amodal boundaries without masks

The second project (Sun et al., 2020) uses CompNets to find the shape of an object. Two boundaries can be asked for. The modal boundary is the outline of the part of the object that you can see. The amodal boundary is the outline of the whole object, including the parts that are occluded; the amodal mask therefore contains the modal mask.

CompNets have a natural ability to detect object boundaries without any annotated segmentation. They are trained, like deep nets, on bounding boxes, which contain the foreground object and some background. They learn generative models of the object, of the context and of occluders; at each position a log-likelihood ratio test between these models says whether the position is foreground, which gives an estimate of the visible (modal) mask.

Most methods for amodal segmentation assume that the visible segmentations have been annotated during training. CompNets get no visible masks either, so they must estimate the modal mask themselves, and both the modal and the amodal masks are evaluated.

CompNets find modal and amodal boundaries without masks



Figure 28: An example of the two definitions (from CGN p. 41): a horse with a rider. The modal mask (red) covers the visible horse; the amodal mask also covers the parts of the horse hidden by the rider's leg.

A shape prior for each viewpoint completes occluded objects

The amodal boundary cannot be seen, so it must come from prior knowledge of the shape of the object. Sun et al. augment CompNets with a prior for the boundary of the object, conditioned on the mixture component: for each viewpoint, a map that gives the probability that each position of the box belongs to the object.

The prior is learnt by an expectation-maximisation (EM) formulation (Lecture 8), with the unknown segmentation of each training box as the hidden variable. The estimates of the boundaries, both modal and amodal, are initialised from the foreground / background estimate of the CompNet. Then the two steps alternate: the segmentations are estimated with the current prior, and the prior of each component is re-estimated from the segmentations of its images. The figure shows the prior sharpening over two iterations.

At test time the CompNet chooses the component, and the prior of that component predicts the amodal box and mask from the visible part. The prior also improves performance on the modal boundaries.

A shape prior for each viewpoint completes occluded objects

In the figure, (a) shows the cars of one mixture component and (b) the prior of that component at iteration 0 and after two iterations (bright: likely object). (c) is the amodal completion of an occluded car: from the visible box (blue) to the amodal box (green); in the label map below it, blue pixels are object, red occluder and green context.

A shape prior for each viewpoint completes occluded objects

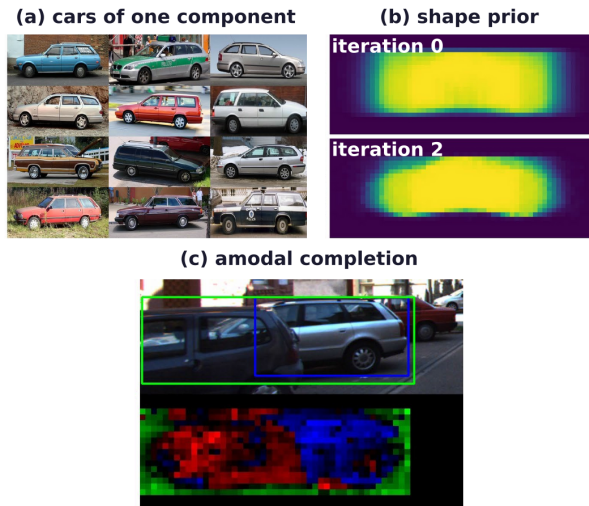


Figure 29: The compositional shape prior (Sun et al., 2020, Fig. 1, from CGN p. 43): (a) one component, (b) its prior, (c) amodal completion.

CompNets segment heavily occluded vehicles best

The test uses KINS, the KITTI instance dataset (Qi et al., 2019), which adds amodal masks to the street scenes of the KITTI benchmark (Karlsruhe Institute of Technology and Toyota Technological Institute; Geiger et al., 2012); Sun et al. use its vehicles. The measure is the mean intersection over union (IoU, in percent) of the predicted and the true amodal masks, by the occluded area of the vehicle.

“Boxes” says whether the methods are trained with modal (visible) or amodal (whole-object) bounding boxes. PCNet-M is the mask network of the partial completion network (Zhan et al., 2020); BBTP learns from boxes with a bounding box tightness prior (Hsu et al., 2019).

method	boxes	0 %	1–30 %	30–60 %	60–90 %	mean
PCNet-M (needs masks)	modal	89.0	75.1	49.1	30.5	60.9
CompNet with prior	modal	72.1	70.4	68.4	52.7	65.9
BBTP	amodal	73.0	70.3	66.6	64.7	68.7
CompNet with prior	amodal	77.4	74.9	78.1	76.3	76.7

CompNets segment heavily occluded vehicles best

PCNet-M also needs ground-truth masks of the objects; BBTP and the CompNet need only the boxes.

With modal boxes, PCNet-M, which was trained with masks, is much better on lightly occluded vehicles, and the CompNet is better from 30 % occlusion on: at 60 to 90 % it reaches 52.7 against 30.5. With amodal boxes the CompNet is best at every level, and its performance hardly falls with the occluded area.

This is an example of multi-task consistency: one model classifies the object and estimates its modal and amodal shape, so the answers come from one interpretation of the image and agree with each other. It is also an example of learning efficiency: the CompNet requires only object annotations (bounding boxes) but performs well on other visual tasks.

CompNets segment heavily occluded vehicles best

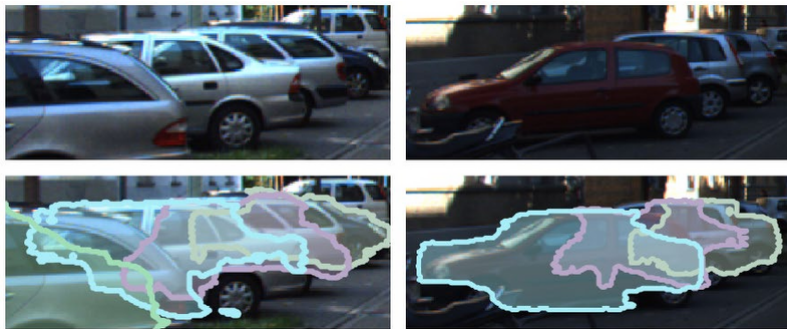


Figure 30: Two examples from KINS (Sun et al., 2020, from CGN p. 43): street scenes with parked cars (top) and the outlines of the amodal masks of the cars, each in its own colour (bottom).

Multi-object occlusion is hard to resolve purely bottom-up

The third project (Yuan et al., 2021) removes an assumption. CompNets assumed there was a single object in the image, which could be occluded. Often two or more objects are present, with one object partly occluding another, as with cars parked in a row. The task is to detect and classify both objects, segment them, and determine the boundary between them: instance-level segmentation.

This visual task is hard to do purely bottom-up. Where the two boxes overlap, each object's CompNet sees features of a car and may claim them; the evidence at one position cannot say which car it belongs to.

Multi-object occlusion is hard to resolve purely bottom-up

The method of Yuan et al. combines two processes:

- ▶ bottom-up, a CompNet for each detected box makes hypotheses for the partial segmentation of its object: likelihood maps that assign each position to the object (blue), the occluder (red) or the context (green);
- ▶ top-down, the occlusion reasoning module (ORM) finds which interpretation of the whole image is most consistent with all the hypotheses.

As in the other projects, only bounding boxes are annotated.

Multi-object occlusion is hard to resolve purely bottom-up

The task relates to CAPTCHAs: the distorted, overlapping letters that websites use to tell people from programs. Reading them requires deciding which letter each stroke belongs to, where the letters overlap. George et al. (2017) broke text CAPTCHAs with a generative vision model that parses the image into letters, a related idea: explain all the evidence by objects, and let each piece of evidence belong to one object.

In the figure, each box (car 1 behind, car 2 in front) goes through a CompNet, which produces a likelihood map (blue object, red occluder, green context; the yellow frame is the overlap with the other box), a class label and the masks of object, context and occluder. The ORM receives the outputs for both boxes and resolves them jointly.

Multi-object occlusion is hard to resolve purely bottom-up

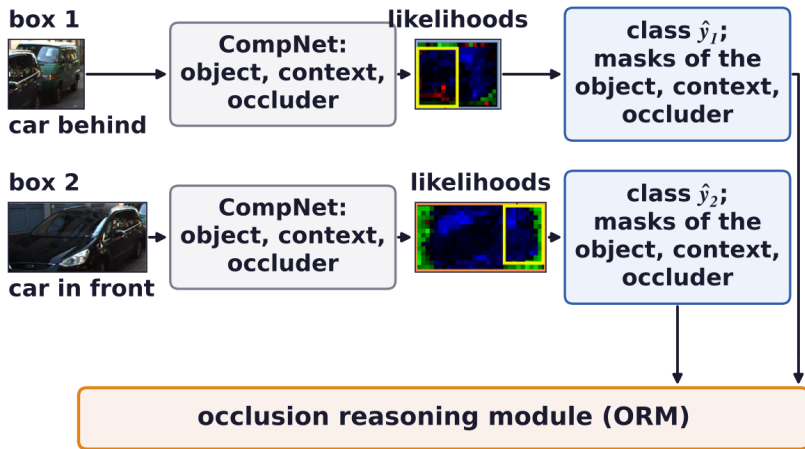


Figure 31: Bottom-up hypotheses and top-down reasoning (Yuan et al., 2021, Fig. 2, redrawn; photographs and maps from CGN p. 48).

The ORM resolves conflicts and recovers the occlusion order

The ORM works in steps. (I) Segmentation conflict: the conflict C is the set of pixels classified as foreground by both object models; it lies in the overlap of the two boxes. (II) Pixel-level competition: each conflict pixel i is given to the model that explains it best,

$$z_{i,1} = 1 \quad \text{if} \quad p(f_i=F, \hat{y}_1) > \max\{p(f_i=F, \hat{y}_2), p(f_i=O)\},$$

where $p(f_i=F, \hat{y}_k)$ is the likelihood that pixel i is foreground of object k with class $\hat{y}_k$, and $p(f_i=O)$ that it is an occluder; $z_{i,2}$ is defined in the same way. These are the paper's symbols: here f_i is the label of a pixel, F foreground, O occluder, and $z_{i,k}$ the assignment of a pixel to object k .

The ORM resolves conflicts and recovers the occlusion order

(III) Pixel reassignment: the masks are updated with these decisions. (IV) Order recovery: the object that wins more of the conflict pixels is in front,

$$R(I_1, I_2) = \begin{cases} 1 & \text{if } \sum_{i \in C} z_{i,1} > \sum_{i \in C} z_{i,2}, \\ -1 & \text{otherwise,} \end{cases}$$

with I_1 and I_2 the two instances, and $R = 1$ meaning that object 1 occludes object 2. (V) The output: amodal masks and an occlusion order. With many objects the pairwise orders form a graph, and the steps are iterated.

The ORM resolves conflicts and recovers the occlusion order

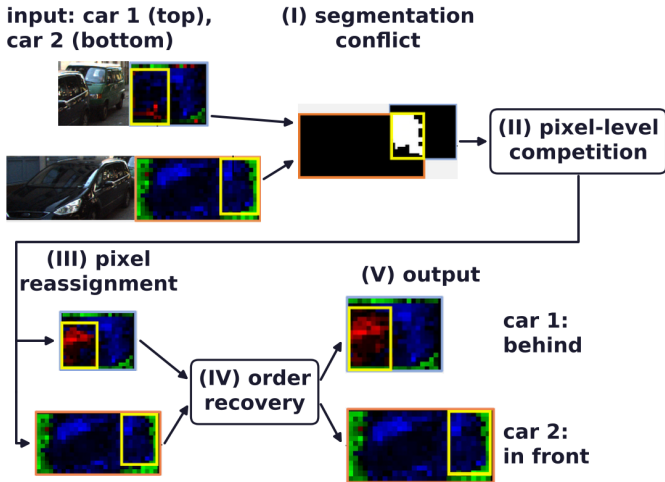


Figure 32: The steps of the ORM (Yuan et al., 2021, Fig. 3, from CGN p. 48, relabelled). Light-blue frames: car 1; orange frames: car 2. (I) Conflict pixels (white) in the overlap of the boxes (yellow). (III), (V) car 2 wins the overlap, which turns red (occluder) in car 1's map.

Reasoning about occlusion improves the instance masks

The method is evaluated on KINS and on an occlusion challenge: KITTI cars cropped by their masks and placed on random backgrounds, in three scenarios: two cars, four cars, and two cars with occluders of unknown classes. The measure is the paper's mean IoU over all occlusion levels (for some rows it differs from the plain average of the four levels). Mask R-CNN (a region-based DCNN trained with ground-truth masks; He et al., 2017) is given for comparison.

mean IoU (%)	2 objects	4 objects	2 + unknown
modal: Mask R-CNN	82.3	78.6	76.7
modal: CompNet	66.9	56.0	63.6
modal: + ORM	73.1	65.3	65.8
amodal: CompNet	76.7	74.8	76.5
amodal: + ORM	79.7	79.5	76.6

The ORM improves every column, most for the modal masks with four cars, where overlaps are most frequent; with unknown occluders the amodal gain is small. Mask R-CNN, trained with masks, is better on the modal masks, here and on KINS.

Reasoning about occlusion improves the instance masks

The model needs bottom-up and top-down processing. It is simpler than the methods used for CAPTCHAs, and it needs extending to harder conditions: more objects, more classes, and real scenes in which the boxes themselves are uncertain.

The figure compares, for the occlusion challenge and for KINS, the input with its boxes, the masks of the CompNet alone, the masks after the ORM, and the ground truth. The ORM corrects the masks where the CompNets of two boxes claimed the same pixels.

Reasoning about occlusion improves the instance masks

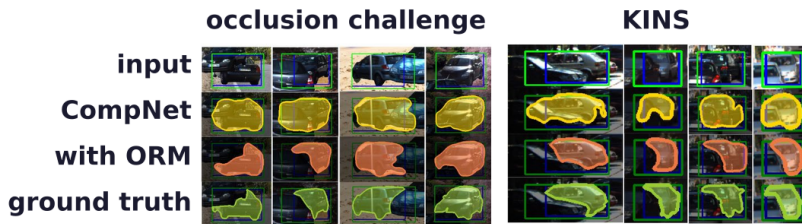


Figure 33: Instance masks (Yuan et al., 2021, Fig. 5, from CGN p. 49). Columns: examples from the occlusion challenge and from KINS. Rows: the input with its boxes, the CompNet masks (yellow), the masks after the ORM (orange), the ground truth (green).

Summary

- ▶ Analysis by synthesis, approximately: CompNets are generative models of deep-network features.
- ▶ Deep nets trained without occlusion fail under occlusion; augmentation closes only part of the gap.
- ▶ A CompNet mixes viewpoints of spatial mixtures of vMF kernels; the kernels resemble part detectors.
- ▶ Inference is a feed-forward network; kernels and mixtures are trained end to end by backpropagation.
- ▶ An outlier model competes with the object model at every position and so localizes the occluders.
- ▶ Separating context from object, and voting for the box, makes detection robust to occlusion.
- ▶ CompNets resist patch attacks out of the box, but need finer detail for fine-grained classes.
- ▶ Shape priors give amodal boundaries; occlusion reasoning orders several overlapping objects.

Summary

The common thread is that knowledge about how images arise is put into the architecture. Objects are made of parts (the vMF kernels and the mixture weights), parts can be hidden (the outlier model), objects appear in contexts that should not dominate (the context-aware model), and objects have shapes (the priors) and occlude each other in an order (the ORM). Each piece is a small generative model with a probabilistic meaning, and each gives an output that can be inspected: which parts were found, where the occluders are, which object is in front.

The limits are those of the approximation. The model is generative on features, not on pixels, so it inherits the backbone's invariances and loses fine detail; it uses a rigid 2D template per viewpoint, so it struggles with articulated objects and classes of very varied appearance. 3D models of objects, and the top-down use of finer features, are the natural next steps.

References

- [1] A. L. Yuille. Compositional generative networks: approximate analysis by synthesis. Course slides (CGN), Johns Hopkins University.
- [2] J. Wang, Z. Zhang, C. Xie, J. Zhu, L. Xie and A. Yuille. Detecting semantic parts on partially occluded objects. *BMVC*, 2017.
- [3] A. Yuille and D. Kersten. Vision as Bayesian inference: analysis by synthesis? *Trends in Cognitive Sciences*, 10(7):301–308, 2006.
- [4] A. Kortylewski, J. He, Q. Liu and A. L. Yuille. Compositional convolutional neural networks: a deep architecture with innate robustness to partial occlusion. *CVPR*, 8940–8949, 2020.
- [5] A. Kortylewski, Q. Liu, A. Wang, Y. Sun and A. Yuille. Compositional convolutional neural networks: a robust and interpretable model for object recognition under occlusion. *International Journal of Computer Vision*, 129(3):736–760, 2021.
- [6] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *ICLR*, 2015.
- [7] A. Banerjee, I. S. Dhillon, J. Ghosh and S. Sra. Clustering on the unit hypersphere using von Mises-Fisher distributions. *Journal of Machine Learning Research*, 6:1345–1382, 2005.

References

- [8] Y. Xiang, R. Mottaghi and S. Savarese. Beyond PASCAL: a benchmark for 3D object detection in the wild. *WACV*, 75–82, 2014.
- [9] A. Kortylewski, Q. Liu, H. Wang, Z. Zhang and A. Yuille. Combining compositional models and deep networks for robust object classification under occlusion. *WACV*, 1333–1341, 2020.
- [10] S. Ren, K. He, R. Girshick and J. Sun. Faster R-CNN: towards real-time object detection with region proposal networks. *NIPS*, 91–99, 2015.
- [11] A. Wang, Y. Sun, A. Kortylewski and A. L. Yuille. Robust object detection under occlusion with context-aware CompositionalNets. *CVPR*, 12645–12654, 2020.
- [12] C. Yang, A. Kortylewski, C. Xie, Y. Cao and A. Yuille. PatchAttack: a black-box texture-based attack with reinforcement learning. *ECCV*, LNCS 12371, 681–698, 2020.
- [13] C. Cosgrove, A. Kortylewski, C. Yang and A. Yuille. Robustness out of the box: compositional representations naturally defend against black-box patch attacks. *arXiv:2012.00558*, 2020.
- [14] F. Croce, M. Andriushchenko, N. D. Singh, N. Flammarion and M. Hein. Sparse-RS: a versatile framework for query-efficient sparse black-box adversarial attacks. *AAAI*, 36(6):6437–6445, 2022.

References

- [15] J. Stallkamp, M. Schlipsing, J. Salmen and C. Igel. Man vs. computer: benchmarking machine learning algorithms for traffic sign recognition. *Neural Networks*, 32:323–332, 2012.
- [16] T. S. Lee and D. Mumford. Hierarchical Bayesian inference in the visual cortex. *Journal of the Optical Society of America A*, 20(7):1434–1448, 2003.
- [17] Y. Sun, A. Kortylewski and A. Yuille. Weakly-supervised amodal instance segmentation with compositional priors. arXiv:2010.13175v1, 2020.
- [18] L. Qi, L. Jiang, S. Liu, X. Shen and J. Jia. Amodal instance segmentation with KINS dataset. *CVPR*, 3014–3023, 2019.
- [19] A. Geiger, P. Lenz and R. Urtasun. Are we ready for autonomous driving? The KITTI vision benchmark suite. *CVPR*, 3354–3361, 2012.
- [20] X. Zhan, X. Pan, B. Dai, Z. Liu, D. Lin and C. C. Loy. Self-supervised scene de-occlusion. *CVPR*, 3784–3792, 2020.

References

- [21] C.-C. Hsu, K.-J. Hsu, C.-C. Tsai, Y.-Y. Lin and Y.-Y. Chuang. Weakly supervised instance segmentation using the bounding box tightness prior. *Advances in Neural Information Processing Systems 32* (NeurIPS), 2019.
- [22] X. Yuan, A. Kortylewski, Y. Sun and A. Yuille. Robust instance segmentation through reasoning about multi-object occlusion. *CVPR*, 11141–11150, 2021.
- [23] D. George, W. Lehrach, K. Kansky, M. Lázaro-Gredilla et al. A generative vision model that trains with high data efficiency and breaks text-based CAPTCHAs. *Science*, 358(6368):eaag2612, 2017.
- [24] K. He, G. Gkioxari, P. Dollár and R. Girshick. Mask R-CNN. *ICCV*, 2961–2969, 2017.